

ISLAMIC UNIVERSITY OF TECHNOLOGY



COMPUTER NETWORKS LAB
CSE 4512

Lab 09 Report

Submitted by:

Student Name: Rahatut Tahrir Mounota

Student ID: 220041122

Department: Computer Science and Engineering (CSE)

Section: 1B

1. Lab Task

The objective of this lab was to design and implement a simple multi-user TCP chat application named **ChatIUT** using Python socket programming and threading.

The tasks included:

- Developing a TCP chat server (`tcp_chat_server.py`) that:
 - Binds to `0.0.0.0` on port `6006` (or CLI-configured port).
 - Accepts multiple client connections.
 - Broadcasts received messages to all other connected clients.
 - Handles client disconnections gracefully.
 - Logs server activities to `server.log`.
- Developing a TCP chat client (`tcp_chat_client.py`) that:
 - Accepts server IP and port from command line.
 - Uses two threads (send and receive).
 - Allows clean exit using `/quit` or `Ctrl+C`.
- Testing multi-client communication and logging.

2. Procedure

Step 1: Environment Setup

Python 3.x was installed. Two Python scripts were created: `tcp_chat_server.py` and `tcp_chat_client.py`.

Step 2: Server Implementation

- Created a TCP socket using `socket.AF_INET` and `SOCK_STREAM`.

- Bound the server to `0.0.0.0:6006`.
- Used a global client list protected by a threading lock.
- Spawned a new `threading.Thread(..., daemon=True)` for each client.
- Implemented a broadcast function to forward messages.
- Logged all events (startup, connect, disconnect, forwarding) to `server.log`.

Step 3: Client Implementation

- Parsed server IP and port using `argparse`.
- Connected to the server.
- Created two threads:
 - Sender thread (`stdin` $\rightarrow$ server).
 - Receiver thread (server $\rightarrow$ terminal).
- Implemented `/quit` command for clean disconnection.

Step 4: Running the Server

```
python tcp_chat_server.py
```

Step 5: Running Multiple Clients

```
python tcp_chat_client.py 127.0.0.1 6006
```

Step 6: Testing

Messages were sent from one client and verified on all other clients. Clients were exited using `/quit` and checked for proper removal from the server list.

3. Observations & Results

Observation 1: Multi-client Broadcast

When Client A sent a message, Client B and Client C successfully received it.

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.
Hello everyone
█
```

```
(.venv) PS E:\chat server> python tcp_chat_server.py
TCP server listening on 0.0.0.0:6006...
Received from ('127.0.0.1', 50904): Hello everyone
█
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.

Received: Hello everyone
█
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006  
Connected to TCP server on 127.0.0.1:6006...  
Type messages and press Enter. Use /quit to exit.
```

```
Received: Hello everyone  
█
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006  
Connected to TCP server on 127.0.0.1:6006...  
Type messages and press Enter. Use /quit to exit.
```

```
Received: Hello everyone  
Hi  
█
```

```
(.venv) PS E:\chat server> python tcp_chat_server.py  
TCP server listening on 0.0.0.0:6006...  
Received from ('127.0.0.1', 50904): Hello everyone  
Received from ('127.0.0.1', 55985): Hi  
█
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.
Hello everyone

Received: Hi
█
```

Figure 1: Successful Multi-Client Chat Session

Observation 2: Graceful Disconnect

When a client exited using `/quit`, the server removed the client from its active list without crashing or producing broken pipe errors.

```
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.

Received: Hello everyone
Hi
/quit
Connection closed.
```

Figure 2: Client disconnect

Observation 3: Logging

The `server.log` file correctly recorded:

- Server startup
- Client connections
- Message forwarding

- Client disconnections

```

server.log
1  2026-03-02 21:53:17,415 INFO: Server started on 0.0.0.0:6006
2  2026-03-02 21:53:31,715 INFO: Client connected: ('127.0.0.1', 50904)
3  2026-03-02 21:53:38,643 INFO: Client connected: ('127.0.0.1', 50906)
4  2026-03-02 21:53:53,509 INFO: Client connected: ('127.0.0.1', 50910)
5  2026-03-02 21:53:54,699 INFO: Client disconnected: ('127.0.0.1', 50910)
6  2026-03-02 21:54:04,861 INFO: Client connected: ('127.0.0.1', 55985)
7  2026-03-02 21:54:54,450 INFO: Received from ('127.0.0.1', 50904): Hello everyone
8  2026-03-02 21:54:54,450 INFO: Forwarded message from ('127.0.0.1', 50904) to other clients
9  2026-03-02 21:56:02,358 INFO: Received from ('127.0.0.1', 55985): Hi
10 2026-03-02 21:56:02,358 INFO: Forwarded message from ('127.0.0.1', 55985) to other clients
11 2026-03-02 21:57:34,744 INFO: Client disconnected: ('127.0.0.1', 50904)
12 2026-03-02 21:57:39,849 INFO: Client disconnected: ('127.0.0.1', 50906)
13 2026-03-02 21:57:45,327 INFO: Client disconnected: ('127.0.0.1', 55985)
14

```

Figure 3: Screenshot of server.log showing startup, message forwarding and disconnect events

All required functionalities were successfully implemented and tested.

4. Challenges Faced

- Managing shared client list safely required proper use of `threading.Lock`.
- Handling abrupt client termination without crashing the server required exception handling and cleanup logic.
- Ensuring clean shutdown using daemon threads and event flags required careful synchronization.
- Avoiding errors during broadcast needed removal of dead clients.

Conclusion:

The ChatIUT multi-user TCP chat system was successfully implemented. The server handled multiple concurrent clients using threading, broadcasted messages correctly, logged events, and managed graceful disconnections as required.